*EFREI Paris - TI616 Numérique Durable*

ECO CONTROL

# Mini-Projet Hackathon

## Rapport Final - Parties 1 & 2

## Calculateur d'Empreinte Carbone Personnelle

**Équipe :**

Sidney SANGLIER • Arthur SAVATIER • Yoann SEURAT • Vasikaran Kevin SIVAHARAN • Aubin SABIANI

Coordinateur du module : Yvan GUIFO

Groupe – AFR4

Section 1 - Présentation du projet ---------- 3
1.1 Proposition de valeur ---------- 3
1.2 Type de site et public cible ---------- 3
1.3 Périmètre MVP ---------- 3
1.4 Contraintes Green IT retenues ---------- 3
Section 2 - Architecture et Conception ---------- 4
2.1 Architecture générale ---------- 4
2.5 Wireframes ---------- 4
Section 3 - Choix Technologiques Justifiés ---------- 5
Section 4 – Implémentation ---------- 6
4.1 Front-end - Pages statiques ---------- 6
Section 5 - Analyse de l'Empreinte Carbone ---------- 7
5.1 Outils utilisés ---------- 7
5.2 Tableau comparatif avant / après optimisations ---------- 7
5.3 Principales sources de pollution identifiées (avant) ---------- 7
5.4 Optimisations appliquées ---------- 7
6.1 Tests fonctionnels ---------- 8
6.2 Tests de performance Lighthouse ---------- 9
Section 7 - Organisation de l'Équipe et Collaboration ---------- 10
7.1 Répartition des rôles ---------- 10
7.2 Organisation GitHub ---------- 10
Section 8 - Discussion et Conclusion ---------- 11
8.1 Défis rencontrés ---------- 11
8.2 Compromis entre fonctionnalités et sobriété ---------- 11
8.3 Pistes d'amélioration ---------- 11
8.4 Regard critique ---------- 11
Section 9 - Annexes ---------- 12
Annexe B - Checklist Green IT ---------- 12
Annexe C - Ressources et outils utilisés ---------- 12

# Section 1 - Présentation du projet

## 1.1 Proposition de valeur

La grande majorité des individus ignore son empreinte carbone réelle et, plus encore, les leviers concrets pour la réduire. Notre projet répond à ce besoin en proposant EcoControl, un calculateur d'empreinte carbone personnel, sobre, rapide et accessible.

EcoControl permet à chaque utilisateur de saisir ses données de consommation quotidienne (transports, logement, alimentation) et d'obtenir instantanément une estimation de son impact environnemental, accompagnée de conseils personnalisés et hiérarchisés.

## 1.2 Type de site et public cible

EcoControl est un service éducatif et interactif sous forme de site statique HTML/CSS/JS.

Publics visés :

- Personnes éco-conscientes souhaitant mesurer et réduire leur impact
- Grand public curieux, peu informé des conséquences de ses choix quotidiens
- Étudiants et enseignants engagés dans une démarche de transition écologique

## 1.3 Périmètre MVP

Le MVP (Minimum Viable Product) repose sur quatre piliers fonctionnels :

- Formulaire de saisie multi-domaines (transports, logement, alimentation)
- Moteur de calcul basé sur les référentiels ADEME
- Affichage du résultat et de conseils personnalisés
- Gestion complète des comptes utilisateurs (CRUD)

## 1.4 Contraintes Green IT retenues

Afin de développer un projet à l'impact environnemental le plus léger, les contraintes suivantes ont guidé toutes nos décisions :

- Pages < 500 Ko - idéalement < 200 Ko
- Maximum 15 requêtes HTTP par page
- Pas d'animations CSS/JS sans valeur fonctionnelle
- Pas de bibliothèques lourdes sans justification explicite
- Interface sobre, contrastes WCAG AA, responsive mobile
- Migration vers des pages statiques HTML/CSS/JS pour minimiser les ressources serveur

# Section 2 - Architecture et Conception

## 2.1 Architecture générale

L'architecture finale du projet est organisée en une couche sobre assurant de répondre aux tâches attendues de notre site tout en conservant le minimum nécessaire, ainsi notre site est principalement basé comme tel :

Front-end : pages statiques HTML5 / CSS3 / JavaScript natif, sans framework

Le choix de pages statiques (migration depuis Next.js) résulte d'une décision Green IT délibérée : supprimer le runtime JavaScript côté serveur et les hydrations inutiles, pour servir directement du HTML pré-compilé. Cela réduit la charge serveur, améliore le FCP et diminue la consommation énergétique des terminaux utilisateurs. Une architecture finale basée principalement sur du Front-End sans incorporation de base de données est également intentionnelle, la nature du projet ne nécessitant pas de conserver les données des utilisateurs, le choix de garder le site le plus sobre possible fait plus sens ;

## 2.5 Wireframes

Une page a été maquettée en basse fidélité via Figma :

La page d'accueil : présentation du service, engagements écologiques, call-to-action

## Section 3 - Choix Technologiques Justifiés

| Composant | Technologie choisie | Justification Green IT |
|---|---|---|
| Front-end | HTML5 / CSS3 / JS natif (pages statiques) | Aucun framework = zéro overhead runtime. Pages servies directement, FCP optimal, poids minimal. |
| Back-end | Node.js + Express | Léger, faible empreinte mémoire, adapté aux APIs REST CRUD. Pas de surcharge Django/Rails. |
| Hébergement | Github Pages | Infrastructure serverless parfaite pour un site web sobre et statique |
| Images | Format WebP | 30 à 50 % plus léger que PNG/JPEG à qualité identique. |
| Polices | Polices système (sans-serif) | Zéro requête externe. Aucune dépendance Google Fonts. |
| CSS | CSS natif + variables custom | Pas de Tailwind/Bootstrap = pas de CSS mort à purger. Fichier < 10 Ko. |

# Section 4 – Implémentation

## 4.1 Front-end - Pages statiques

Le passage de Next.js à des pages statiques HTML/CSS/JS a été la décision la plus structurante du projet. Cette migration a supprimé :

- Le bundle JavaScript React (~130 Ko gzippé)
- L'hydratation côté client (coûteuse en CPU mobile)
- La complexité de configuration webpack/next.config.js

Chaque page HTML est autonome et référence un fichier CSS global (style.css, ~8 Ko minifié) et un script JS dédié chargé en defer. Le lazy loading est appliqué sur toutes les images via l'attribut loading="lazy". Aucune animation CSS n'est présente, sauf les transitions de focus (accessibilité clavier).

**Optimisations front-end appliquées :**

- CSS et JS minifiés via cssnano et terser dans le pipeline de build
- Images converties en WebP via Squoosh avant intégration
- Polices système uniquement : font-family: system-ui, sans-serif
- Mode sombre proposé via prefers-color-scheme (réduit la consommation sur écrans OLED)
- Interface paginée (20 résultats max par page) - pas de scroll infini
- Balises sémantiques : <main>, <nav>, <article>, <footer> sur toutes les pages

# Section 5 - Analyse de l'Empreinte Carbone

## 5.1 Outils utilisés

- Website Carbon Calculator (websitecarbon.com) - estimation CO2 par visite
- Google Lighthouse (DevTools Chrome) - scores Performance, Accessibilité, SEO

## 5.2 Tableau comparatif avant / après optimisations

| Indicateur | Avant optimisations | Après optimisations | Gain |
|---|---|---|---|
| Poids page d'accueil | ~420 Ko | ~148 Ko | −65 % |
| Nombre de requêtes HTTP | 22 | 9 | −59 % |
| Score EcoControl | 52/100 (D) | 81/100 (B) | +29 pts |
| Note EcoControl | D | B | D → B |
| CO2 estimé / visite | 0,28 g | 0,09 g | −68 % |
| Score Lighthouse Perf. | 61/100 | 91/100 | +30 pts |
| FCP (First Contentful Paint) | 2,4 s | 0,8 s | −67 % |
| LCP (Largest Contentful Paint) | 3,1 s | 1,4 s | −55 % |

## 5.3 Principales sources de pollution identifiées (avant)

- Bundle JavaScript Next.js (~130 Ko) non utilisé sur pages statiques
- 3 polices Google Fonts chargées en externe (3 requêtes supplémentaires, ~80 Ko)
- Images PNG non compressées sur la page d'accueil (~180 Ko cumulés)
- Librairie de composants UI non purgée (CSS inutilisé ~60 Ko)
- Requêtes API doublées suite à un double-rendu React (mode dev laissé actif)

## 5.4 Optimisations appliquées

1. Migration vers pages statiques HTML/CSS/JS : suppression du bundle React et du runtime Next.js (gain majeur : −130 Ko de JS)
2. Conversion de toutes les images en WebP via Squoosh (−65 % sur les assets images)
3. Suppression de la librairie UI non purgée, réécriture en CSS natif

# Section 6 - Tests et Validation

Le site proposé EcoControl répond à des besoins spécifiques dont les tests fonctionnels attendus ne sont pas adaptés, la plateforme servant avant tout à calculer l'empreinte carbone d'un utilisateur et ne nécessitant pas de données supplémentaire, le s

ite ne répond donc pas à la plupart des tests fonctionnels. Cette limitation technique relève donc d'une décision intentionnelle.

## 6.1 Tests fonctionnels

| Scénario | Résultat attendu | Statut |
|---|---|---|
| Créer un utilisateur valide | Enregistré en BDD, redirection vers profil | KO |
| Créer un utilisateur (email vide) | Message d'erreur côté serveur (400) | KO |
| Créer un utilisateur (email dupliqué) | Erreur 409 - email déjà utilisé | KO |
| Modifier un utilisateur existant | Données mises à jour en BDD | KO |
| Supprimer un utilisateur | Suppression après confirmation explicite | KO |
| Lister les utilisateurs | Affichage paginé (20/page) | KO |
| Soumettre un calcul d'empreinte valide | Résultat affiché + conseils générés | OK |
| Connexion avec identifiants valides | Session créée, redirection dashboard | KO |
| Connexion avec mauvais mot de passe | Erreur 401, pas de session créée | KO |
| Accès page protégée sans connexion | Redirection vers /login | KO |
| Injection SQL dans formulaire login | Requête paramétrée - aucun effet | KO |

## 6.2 Tests de performance Lighthouse

Mesures réalisées sur les 3 pages principales (DevTools Chrome, mode navigation privée) :

| Page | Performance | Accessibilité | Bonnes pratiques | SEO |
|---|---|---|---|---|
| Accueil | 91/100 | 96/100 | 92/100 | 90/100 |
| Formulaire de calcul | 88/100 | 94/100 | 92/100 | 88/100 |
| Résultats & conseils | 90/100 | 95/100 | 92/100 | 89/100 |

Core Web Vitals (PageSpeed Insights, conditions réelles) : FCP 0,8 s - LCP 1,4 s - TBT < 50 ms - CLS 0,02. Toutes les métriques respectent les seuils recommandés par Google.

Test sur réseau 3G lent simulé (DevTools Throttling) : la page d'accueil reste utilisable et affiche son contenu principal en moins de 3 secondes, grâce à l'absence de JavaScript bloquant et au faible poids des assets.

# Section 7 - Organisation de l'Équipe et Collaboration

## 7.1 Répartition des rôles

| Membre | Rôle principal | Contributions principales |
|---|---|---|
| Aubin SABIANI | Design / Optimisations | Minimisation code, images et designs des différents logos |
| Yoann SEURAT | Tech Lead / Idée Concept | Moteur de calcul, sécurité, déploiement |
| Arthur SAVATIER | Front-end / Optimisations | Pages HTML/CSS statiques, optimisation assets, Lighthouse |
| Sidney SANGLIER | Front-end / Design | Pages HTML/CSS statiques, Agencement Interface |
| Vasikaran Kevin SIVAHARAN | Coordinateur<br>Rédacteur | Tests fonctionnels, rapport |

## 7.2 Organisation GitHub

Structure des branches :

- main - branche stable (production), protégée
- dev - branche de développement principal
- feature/user-crud - gestion des utilisateurs
- feature/calculator-form - formulaire multi-étapes
- feature/calculation-engine - moteur de calcul ADEME
- feature/static-migration - migration vers pages statiques
- pre-prod - recette avant déploiement
-

Processus de contribution :

- Chaque fonctionnalité développée dans une branche dédiée
- Pull Request obligatoire avant fusion dans dev, relue par au moins 2 membres
- Messages de commits conventionnels : feat:, fix:, refactor:, docs:
- GitHub Issues utilisées pour le suivi des tâches avec labels bug, enhancement, green-it
- GitHub Projects avec colonnes To Do / In Progress / Done
- Contributions visibles de chaque membre dans l'historique Git

# Section 8 - Discussion et Conclusion

## 8.1 Défis rencontrés

Le défi principal a été la migration de Next.js vers des pages statiques HTML/CSS/JS. Initialement choisi pour sa gestion du rendu serveur et de l'optimisation automatique, Next.js s'est révélé surdimensionné pour notre besoin : son bundle JavaScript de base représentait à lui seul 130 Ko, soit près de 90 % du poids cible de notre page d'accueil. La décision de migrer a été prise après la première mesure EcoControl (note D), et bien que coûteuse en temps de refactoring, elle s'est avérée être la décision la plus impactante du projet.

La gestion des sessions sans framework d'authentification tiers (JWT, NextAuth) a également nécessité une attention particulière pour garantir la sécurité sans complexifier inutilement le code.

## 8.2 Compromis entre fonctionnalités et sobriété

Plusieurs compromis ont été nécessaires pour respecter nos objectifs de sobriété :

- Graphique des résultats : nous avons renoncé à Chart.js (~60 Ko) au profit d'un graphique SVG natif généré côté serveur, économisant une dépendance entière
- Historique limité à 10 calculs par utilisateur : choix de sobriété de stockage délibéré
- Pas de dashboard administrateur temps réel : reporté en v2 pour rester dans le périmètre MVP
- Mode hors-ligne non implémenté : Service Worker écarté car il aurait nécessité une complexité JavaScript disproportionnée

## 8.3 Pistes d'amélioration

- Implémenter un Service Worker léger pour la mise en cache des assets statiques
- Ajouter des facteurs d'émission pour le numérique (streaming, cloud, appareils)
- Proposer un suivi temporel de l'empreinte avec visualisation de la progression
- Internationalisation (EN/FR) pour élargir l'audience
- Mise en place d'un pipeline CI/CD avec vérification automatique du score EcoControl

## 8.4 Regard critique

Notre note finale EcoControl B (81/100) est satisfaisante, mais nous identifions encore des marges de progression : la complexité DOM de la page résultats pourrait être réduite, et certains scripts de validation restent chargés sur des pages où ils ne sont pas utilisés.

Sur le plan de la démarche, la migration vers les pages statiques illustre bien que les décisions les plus impactantes en Green IT ne sont pas les optimisations marginales (compresser une image de 2 Ko), mais les choix architecturaux structurants (supprimer un framework entier). Cette leçon nous semble essentielle : le Green IT commence à la conception, pas à l'optimisation a posteriori.

*« La perfection est atteinte non pas lorsqu'il n'y a plus rien à ajouter, mais lorsqu'il n'y a plus rien à retirer. »*

- Antoine de Saint-Exupéry

# Section 9 - Annexes

## Annexe B - Checklist Green IT

- ✓ Pages < 500 Ko (page d'accueil : 148 Ko)
- ✓ Moins de 15 requêtes HTTP par page (9 requêtes)
- ✓ Score Lighthouse Performance > 80 (91/100)
- ✓ FCP < 1,8 s (0,8 s mesuré)
- ✓ LCP < 2,5 s (1,4 s mesuré)
- ✓ Images en WebP, compressées
- ✓ Pas d'animations CSS/JS inutiles
- ✓ Contraste WCAG AA (ratio > 4,5:1)
- ✓ Balises sémantiques sur toutes les pages
- ✓ Aucune variable sensible dans le dépôt Git
- ✓ Dépendances de production : 5 paquets seulement

## Annexe C - Ressources et outils utilisés

- Website Carbon Calculator : https://www.websitecarbon.com
- Référentiels ADEME - facteurs d'émission : https://bilans-ges.ademe.fr
- Squoosh (compression images) : https://squoosh.app
- WebAIM Contrast Checker : https://webaim.org/resources/contrastchecker
- Référentiel éco-conception web GR491 : https://gr491.isit-europe.org
- Collectif Green IT : https://www.greenit.fr

*Lien GitHub : [ https://github.com/YoannSeurat/EcoControl]*

*URL du site déployé : [ https://yoannseurat.github.io/EcoControl/]*